

Database Management System

Trigger, Transaction, Cursor and Temporary Table

Dr. Tran Anh Tuan,

TRIGGER

- - Trigger là một đối tượng chứa một tập các câu lệnh SQL và tập các câu lệnh này sẽ được thực thi khi trigger được gọi.
- - Các trigger lại được “gọi” tự động khi xảy ra những giao dịch làm thay đổi dữ liệu trong các bảng.
- - Mỗi một trigger được tạo ra và gắn liền với một bảng nào đó trong cơ sở dữ liệu. Khi dữ liệu trong bảng bị thay đổi (tức là khi bảng chịu tác động của các câu lệnh INSERT, UPDATE hay DELETE) thì trigger sẽ được tự động kích hoạt.

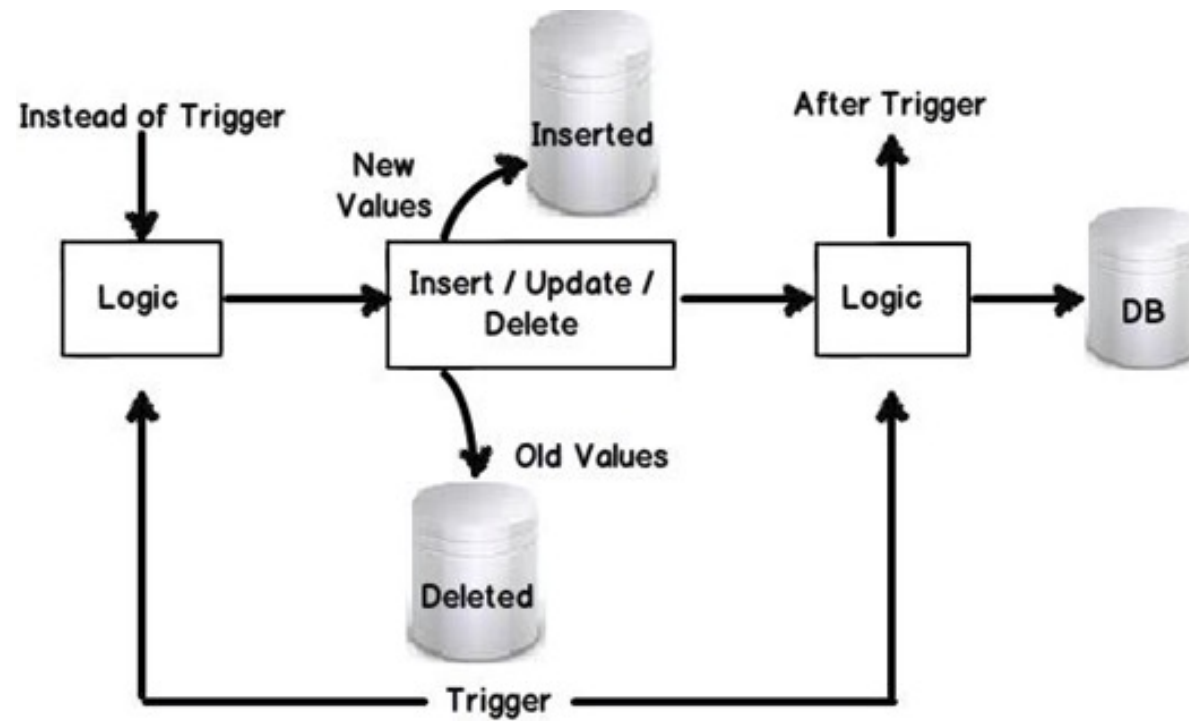


TRIGGER



- Sử dụng trigger một cách hợp lý trong cơ sở dữ liệu sẽ có tác động rất lớn trong việc tăng hiệu năng của cơ sở dữ liệu. Các trigger thực sự hữu dụng với những khả năng sau:
 - Một trigger có thể nhận biết, ngăn chặn và huỷ bỏ được những thao tác làm thay đổi trái phép dữ liệu trong cơ sở dữ liệu.
 - Các thao tác trên dữ liệu (xoá, cập nhật và bổ sung) có thể được trigger phát hiện ra và tự động thực hiện một loạt các thao tác khác trên cơ sở dữ liệu nhằm đảm bảo tính hợp lệ của dữ liệu.
 - Thông qua trigger, ta có thể tạo và kiểm tra được những mối quan hệ phức tạp hơn giữa các bảng trong cơ sở dữ liệu mà bản thân các ràng buộc không thể thực hiện được.

TRIGGER



SQL Statement

```
UPDATE t1 SET ...;
```

Fires the
UPDATE_T1
Trigger

UPDATE_T1 Trigger

```
BEFORE UPDATE ON t1  
FOR EACH ROW  
BEGIN  
  .  
  .  
  INSERT INTO t2 VALUES (...);  
  .  
  .  
END;
```

Fires the
INSERT_T2
Trigger

INSERT_T2 Trigger

```
BEFORE INSERT ON t2  
FOR EACH ROW  
BEGIN  
  .  
  .  
  INSERT INTO ... VALUES (...);  
  .  
  .  
END;
```

etc.



insert into clients (**first_name**, **last_name**, **city**, **account_type**)
values ('Ankur','Malhotra','Mumbai','key_account')

„clients”

id	first_name	last_name	city	account_type
1	Barun	Rao	Achalpur	key_account
2	Aakash	Rangan	Achhnera	normal_account
3	Aadit	Gupta	Bangalore	normal_account
4	Aamir	Malhotra	Agra	normal_account
5	Ankur	Malhotra	Mumbai	key_account

temporary table „**inserted**”

id	first_name	last_name	city	account_type
5	Ankur	Malhotra	Mumbai	key_account

```
CREATE TRIGGER trg_test_insert ON clients
FOR INSERT
AS
begin
    select * from inserted
end
```

TRIGGER



- Một trigger là một đối tượng gắn liền với một bảng và được tự động kích hoạt khi xảy ra những giao dịch làm thay đổi dữ liệu trong bảng. Định nghĩa một trigger bao gồm các yếu tố sau:
- Trigger sẽ được áp dụng đối với bảng nào?
- Trigger được kích hoạt khi câu lệnh nào được thực thi trên bảng:

```
CREATE TRIGGER tên_trigger
ON tên_bảng
FOR {[INSERT][,][UPDATE][,][DELETE]}
AS
[IF UPDATE(tên_cột)
[AND UPDATE(tên_cột)][OR UPDATE(tên_cột)]...]
các_câu_lệnh_của_trigger
```

TRIGGER



- Ta định nghĩa các bảng như sau:
 - Bảng MATHANG lưu trữ dữ liệu về các mặt hàng:

```
CREATE TABLE mathang
( mahang    NVARCHAR(5)    PRIMARY KEY, /*mã hàng*/
  tenhang    NVARCHAR(50)    NOT NULL, /*tên hàng*/
  soluong    INT, /*số lượng hàng hiện có*/ )
```


TRIGGER



- Ta định nghĩa các bảng như sau:
 - Bảng NHATKYBANHANG lưu trữ thông tin về các lần bán hàng

```
CREATE TABLE    nhatkybanhang
( stt INT      IDENTITY    PRIMARY KEY,
  ngay DATETIME, /*ngày bán hàng*/
  nguoimua NVARCHAR(30),/*tên người mua hàng*/
  mahang NVARCHAR(5) /*mã mặt hàng được bán*/ FOREIGN KEY REFERENCES mathang(mahang),
  soluong INT, /*giá bán hàng*/
  giaban MONEY /*số lượng hàng được bán*/
)
```

TRIGGER



- Câu lệnh dưới đây định nghĩa trigger *trg_nhatkybanhang_insert*. Trigger này có chức năng tự động giảm số lượng hàng hiện có khi một mặt hàng nào đó được bán (tức là khi câu lệnh INSERT được thực thi trên bảng NHATKYBANHANG).

```
CREATE TRIGGER trg_nhatkybanhang_insert  
ON nhatkybanhang  
FOR INSERT
```

```
AS  
  
UPDATE mathang  
SET mathang.soluong=mathang.soluonginserted.soluong  
FROM mathang INNER JOIN inserted  
ON mathang.mahang=inserted.mahang
```

TRIGGER



- Với trigger vừa tạo ở trên, nếu dữ liệu trong bảng MATHANG là:

MAHANG	TENHANG	SOLUONG
H1	Xà phòng	30
H2	Kem đánh răng	45

- thì sau khi ta thực hiện câu lệnh:

```
INSERT INTO nhattybanhang (ngay, nguoi mua, mahang, soluong, giaban)
VALUES ('5/5/2004', 'Tran Ngoc Thanh', 'H1', 10, 5200)
```

- dữ liệu trong bảng MATHANG sẽ như sau:

MAHANG	TENHANG	SOLUONG
H1	Xà phòng	20
H2	Kem đánh răng	45

TRIGGER



- Chuẩn SQL định nghĩa hai bảng logic INSERTED và DELETED để sử dụng trong các trigger. Cấu trúc của hai bảng này tương tự như cấu trúc của bảng mà trigger tác động. Dữ liệu trong hai bảng này tùy thuộc vào câu lệnh tác động lên bảng làm kích hoạt trigger; cụ thể trong các trường hợp sau:
 - Khi câu lệnh DELETE được thực thi trên bảng, các dòng dữ liệu bị xoá sẽ được sao chép vào trong bảng DELETED. Bảng INSERTED trong trường hợp này không có dữ liệu.
 - Dữ liệu trong bảng INSERTED sẽ là dòng dữ liệu được bổ sung vào bảng gây nên sự kích hoạt đối với trigger bằng câu lệnh INSERT. Bảng DELETED trong trường hợp này không có dữ liệu.
 - Khi câu lệnh UPDATE được thực thi trên bảng, các dòng dữ liệu cũ chịu sự tác động của câu lệnh sẽ được sao chép vào bảng DELETED, còn trong bảng INSERTED sẽ là các dòng sau khi đã được cập nhật

TRIGGER



- **Sử dụng mệnh đề IF UPDATE trong trigger**

- Thay vì chỉ định một trigger được kích hoạt trên một bảng, ta có thể chỉ định trigger được kích hoạt và thực hiện những thao tác cụ thể khi việc thay đổi dữ liệu chỉ liên quan đến một số cột nhất định nào đó của cột. Trong trường hợp này, ta sử dụng mệnh đề

```
CREATE TRIGGER trg_nhatkybanhang_update_soluong
ON nhatkybanhang
FOR UPDATE
AS
IF UPDATE(soluong)
    UPDATE mathang
    SET mathang.soluong = mathang.soluong -
    (inserted.soluong-deleted.soluong)
FROM (deleted INNER JOIN inserted ON
    deleted.stt = inserted.stt) INNER JOIN mathang
ON mathang.mahang = deleted.mahang
```

TRIGGER



- **Sử dụng mệnh đề IF UPDATE trong trigger**

```
CREATE TRIGGER trg_nhatkybanhang_update_soluong
ON nhatkybanhang
FOR UPDATE
AS
IF UPDATE(soluong)
    UPDATE mathang
    SET mathang.soluong = mathang.soluong -
    (inserted.soluong-deleted.soluong)
    FROM (deleted INNER JOIN inserted ON
        deleted.stt = inserted.stt) INNER JOIN mathang
    ON mathang.mahang = deleted.mahang
```

TRIGGER



- Với trigger ở ví dụ trên, câu lệnh:

```
UPDATE nhatkybanhang  
SET soluong=soluong+20  
WHERE stt=1
```

sẽ kích hoạt trigger ứng với mệnh đề IF UPDATE (soluong)

Tuy nhiên câu lệnh

```
UPDATE nhatkybanhang  
SET nguoiimua='Mai Hữu Toàn'  
WHERE stt=3
```

lại không kích hoạt trigger này.

TRIGGER



- Mệnh đề IF UPDATE có thể xuất hiện nhiều lần trong phần thân của trigger. Khi đó, mệnh đề IF UPDATE nào đúng thì phần câu lệnh của mệnh đề đó sẽ được thực thi khi trigger được kích hoạt.

```
CREATE TRIGGER trg_R_test
ON R
FOR UPDATE
AS
    IF UPDATE(A)
        Print 'A updated'
    IF UPDATE(C)
        Print 'C updated'
```

Câu lệnh

```
UPDATE R SET A=100 WHERE A=1
```

sẽ kích hoạt trigger và cho kết quả là: A updated
và câu lệnh:

```
UPDATE R SET C=100 WHERE C=2
```

cũng kích hoạt trigger và cho kết quả là: C updated

TRIGGER



- Một trigger có khả năng nhận biết được sự thay đổi về mặt dữ liệu trên bảng dữ liệu, từ đó có thể phát hiện và huỷ bỏ những thao tác không đảm bảo tính toàn vẹn dữ liệu. Trong một trigger, để huỷ bỏ tác dụng của câu lệnh làm kích hoạt trigger, ta sử dụng câu lệnh

- ROLLBACK TRANSACTION

- Nếu trên bảng MATHANG, ta tạo một trigger như sau:

```
CREATE TRIGGER trg_mathang_delete
ON mathang
FOR DELETE
AS
ROLLBACK TRANSACTION
```

- Thì câu lệnh DELETE sẽ không thể có tác dụng đối với bảng mathang

TRIGGER

```
CREATE TRIGGER trg_nhatkybanhang_insert
ON NHATKYBANHANG
FOR INSERT
AS
DECLARE @sl_co int /* Số lượng hàng hiện có */
DECLARE @sl_ban int /* Số lượng hàng được bán */
DECLARE @mahang nvarchar(5) /* Mã hàng được bán */
SELECT @mahang=mahang, @sl_ban=soluong
FROM inserted
SELECT @sl_co = soluong
FROM mathang where mahang=@mahang
/*Nếu số lượng hàng hiện có nhỏ hơn số lượng bán thì |
        huỷ bỏ thao tác bổ sung dữ liệu */
```

- - Trigger dưới đây được kích hoạt khi câu lệnh INSERT được sử dụng để bổ sung một bản ghi mới cho bảng NHATKYBANHANG.
- - Trong trigger này kiểm tra điều kiện hợp lệ của dữ liệu là số lượng hàng bán ra phải nhỏ hơn hoặc bằng số lượng hàng hiện có. Nếu điều kiện này không thoả mãn thì huỷ bỏ thao tác bổ sung dữ liệu.

TRIGGER

```
CREATE TRIGGER trg_nhatkybanhang_insert
ON NHATKYBANHANG
FOR INSERT
AS
DECLARE @sl_co int /* Số lượng hàng hiện có */
DECLARE @sl_ban int /* Số lượng hàng được bán */
DECLARE @mahang nvarchar(5) /* Mã hàng được bán */
SELECT @mahang=mahang, @sl_ban=soluong
FROM inserted

SELECT @sl_co = soluong
FROM mathang where mahang=@mahang

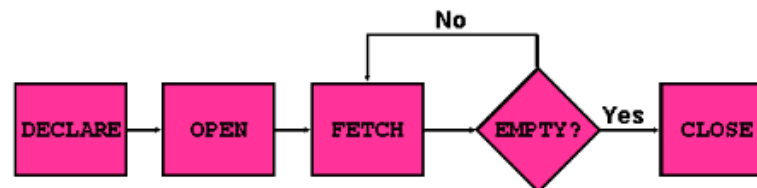
/*Nếu số lượng hàng hiện có nhỏ hơn số lượng bán thì |
        huỷ bỏ thao tác bổ sung dữ liệu */
```

```
IF @sl_co<@sl_ban
        ROLLBACK TRANSACTION
/* Nếu dữ liệu hợp lệ thì giảm số lượng hàng hiện có */
ELSE
        UPDATE mathang
        SET soluong=soluong-@sl_ban
        WHERE mahang=@mahang
```

CURSOR



- - Khi cần duyệt qua từng dòng dữ liệu của một bảng để thao tác với chúng
- - Chỉ dùng khi không còn cách nào khác để cho ra kết quả tương tự, hoặc trong khả năng của bạn, chưa thể tìm cách giải quyết, chỉ chiếm dưới 5% thôi, tùy trường hợp cụ thể, còn hầu hết các trường hợp còn lại, đều có thể sử dụng các câu lệnh T-SQL khác để xử lý và nên sử dụng cách khác vì vấn đề tốc độ thực thi của cursor trong hầu hết trường hợp đều chậm hơn vì chúng thường sử dụng nhiều tài nguyên SQL Server
- - Hãy cân nhắc khi câu lệnh của bạn tác động đến bảng có số lượng lớn records, tốt nhất là khi bạn chắc chắn là bạn tác động đến không nhiều dữ liệu hiện tại cũng như sự phát triển số lượng dữ liệu sau này và không ảnh hưởng đáng kể đối với tốc độ của ứng dụng khi thực hiện tác vụ nào đó



```
DECLARE @CustomerId INT
```

Declare Variable for holding records

```
DECLARE Cursor1 CURSOR READ_ONLY  
FOR
```

Declare the Cursor

```
SELECT CustomerId  
FROM Customers
```

The Select Query

```
OPEN Cursor1
```

Open the Cursor

```
FETCH NEXT FROM Cursor1 INTO  
@CustomerId
```

Fetch the First record into the variable

```
WHILE @@FETCH_STATUS = 0  
BEGIN
```

Loop until records being fetched

```
    PRINT @CustomerId
```

Process the record

```
    FETCH NEXT FROM Cursor1 INTO  
    @CustomerId
```

Fetch the next record

```
END
```

Close the Cursor

```
CLOSE Cursor1
```

Deallocate the Cursor

```
DEALLOCATE Cursor1
```

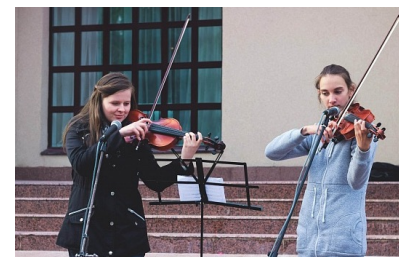
CURSOR



- - Việc định nghĩa biến có kiểu cursor trong Transaction-SQL là việc chỉ định đến những dòng dữ liệu có bên trong các bảng dữ liệu nào mà biến sẽ tham chiếu đến. Thông thường trong lệnh định nghĩa biến có kiểu cursor bên dưới sẽ chỉ định ra loại của cursor cho các mục đích sử dụng về sau này thuận tiện hơn.

```
DECLARE Tên_cursor CURSOR  
[LOCAL | GLOBAL]  
[FORWARD_ONLY | SCROLL]  
[STATIC | DYNAMIC | KEYSET]  
[READ_ONLY | SCROLL_LOCK]  
FOR Câu_lệnh_SELECT  
[FOR UPDATE [OF Danh_sách_cộtcn]]
```

CURSOR



- **Tên cursor** : tên của biến kiểu cursor.
- **Từ khóa LOCAL\GLOBAL** : dùng chỉ định phạm vi hoạt động của biến cursor hoặc là cục bộ (local) bên trong một thủ tục, lô (batch) các lệnh, một trigger hoặc là toàn cục (global) bên trong một kết nối. Một biến cursor có tính toàn cục sẽ được phép tham chiếu trong bất kỳ thủ tục nào của kết nối tạo ra biến cursor đó.
- **Từ khóa FORWARD ONLY** : dùng chỉ định việc đọc dữ liệu trong cursor chỉ theo chiều đi tới mà thôi (duyệt từ mẫu tin đầu tiên đến mẫu tin cuối cùng).
- **Từ khóa SCROLL** : dùng chỉ định việc đọc dữ liệu trong cursor được phép di chuyển tới lui, qua lại các dòng mẫu tin bên trong cursor tùy thích.
- **Từ khóa STATIC** : dùng chỉ định dữ liệu đọc bên trong cursor là tĩnh. Khi đó nếu những người dùng khác có các thay đổi ở bên dưới dữ liệu gốc (base table) thì các thay đổi đó sẽ không được cập nhật tự động trong dữ liệu của cursor. Bởi vì khi đó dữ liệu trong cursor chính là dữ liệu của một bảng tạm đã được hệ thống sao chép và lưu trữ trong cơ sở dữ liệu tempdb của hệ thống khi định nghĩa cursor.

CURSOR



- **Từ khóa DYNAMIC** : dùng chỉ định dữ liệu bên trong cursor là động. Khi đó việc cập nhật dữ liệu trong bảng cơ sở (base table) bởi những người dùng khác sẽ được cập nhật tự động trong dữ liệu cursor có kiểu là DYNAMIC
- **Từ khóa READ_ONLY** : dùng chỉ định dữ liệu bên trong cursor là chỉ đọc nhằm hạn chế việc sửa đổi dữ liệu bên trong cursor. Khi khai báo cursor với kiểu dữ liệu là tĩnh (STATIC) thì dữ liệu trong cursor xem như là chỉ đọc.
- **Từ khóa SCROLL_LOCK** : dùng chỉ định hệ thống Microsoft SQL Server tự động khóa các dòng mẫu tin cần phải thay đổi giá trị hoặc bị hủy bỏ bên trong bảng nhằm đảm bảo các hành động cập nhật luôn luôn thành công.
 - **Câu lệnh SELECT** : dùng để chỉ đến các cột có bên trong bảng mà bạn cần đọc dữ liệu. Câu lệnh SELECT trong cursor không thể chứa các mệnh đề : INTO, COMPUTE, COMPUTE BY.
 - **Danh sách các cột cập nhật** : chỉ định danh sách tên các cột sẽ được phép thay đổi giá trị trong cursor. Mặc định tất cả các cột trong mệnh đề SELECT sẽ được phép thay đổi giá trị nếu dữ liệu cursor không phải là chỉ đọc.

CURSOR



- Để định nghĩa một biến cursor chứa toàn bộ các dòng dữ liệu bên trong bảng VATTU, các dòng dữ liệu trong cursor cho phép được cập nhật. Bạn sử dụng lệnh khai báo biến cursor như sau :

```
DECLARE cur_Vattu CURSOR  
DYNAMIC  
FOR  
SELECT * FROM VATTU
```

- Để định nghĩa một biến cursor chứa toàn bộ các dòng dữ liệu bên trong bảng NHACC, các dữ liệu trong cursor chỉ được phép đọc và việc đọc dữ liệu trong cursor chỉ theo một chiều tới. Bạn sử dụng lệnh khai báo biến cursor như sau :

```
DECLARE cur_Nhacc CURSOR  
FORWARD_ONLY  
STATIC  
READ_ONLY  
FOR  
SELECT * FROM NHACC
```

CURSOR



- **Mở cursor**

- Để có thể đọc được các dòng dữ liệu bên trong cursor trước tiên bạn cần phải mở cursor ra bằng lệnh OPEN. Hoạt động bên trong của lệnh này thực ra là hệ thống sẽ thực hiện câu lệnh truy vấn SELECT đã được chỉ định trong lệnh định nghĩa biến cursor trước đó.

OPEN Tên_cursor

Ví dụ :

Để mở các cursor đã định nghĩa ở các ví dụ trên. Bạn sử dụng lệnh OPEN như sau :

OPEN cur_Vattu

Hoặc

OPEN cur_Nhacc

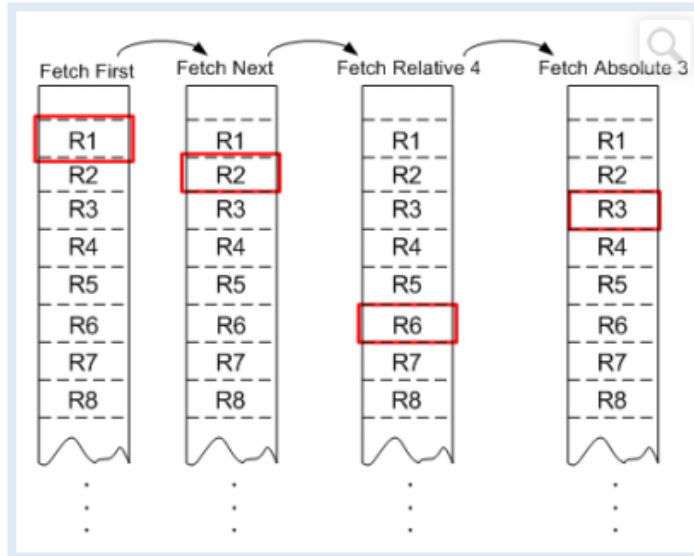
CURSOR



- **Đọc và xử lý dữ liệu trong cursor**

Cú pháp :

```
FETCH [NEXT | PRIOR | FIRST | LAST | ABSOLUTE n | RELATIVE n]  
FROM Tên_cursor  
[INTO Danh_sách_biến]
```



Từ khóa RELATIVE : dùng để chỉ định việc đọc dữ liệu tại một dòng tương đối so với dòng dữ liệu hiện hành. Với n là một số nguyên có thể dương hoặc âm để chỉ định việc đọc theo chiều tới hoặc lui so với dòng dữ liệu hiện hành.

Từ khóa ABSOLUTE : dùng để chỉ định dữ liệu chính xác thứ n bên trong cursor. Với n là số nguyên dương dùng chỉ định việc đọc dữ liệu tại dòng thứ n được đếm từ dòng đầu tiên, với n là số nguyên âm dùng chỉ định việc đọc dữ liệu tại dòng thứ n được đếm ngược từ dòng cuối cùng trở lên.

CURSOR

- @@FETCH_STATUS dùng để kiểm tra tình trạng đọc dữ liệu là thành công hay thất bại. Giá trị của biến trả về 0 khi việc đọc dữ liệu là thành công.
- Ví dụ : Để đọc dữ liệu cursor bảng VATTU chỉ lọc các vật tư là Tivi. Bạn sử dụng các lệnh như sau :

Kết quả trả về :

MAVTU	TENVTU	DVTINH	PHANTRAM
TV14	Tivi Sony 14 inches	Cái	20
(1 row(s) affected)			
MAVTU	TENVTU	DVTINH	PHANTRAM
TV21	Tivi Sony 21 inches	Cái	15
(1 row(s) affected)			
MAVTU	TENVTU	DVTINH	PHANTRAM
TV29	Tivi Sony 29 inches	Cái	15
(1 row(s) affected)			



```
--1. Khai báo biến cursor
DECLARE cur_Vattu CURSOR
KEYSET
FOR
    SELECT * FROM VATTU
        WHERE MAVTU LIKE "TV%"
        ORDER BY MAVTU

--2. Mở cursor
FETCH NEXT FROM cur_Vattu

--3. Đọc dữ liệu
FETCH NEXT FROM cur_Vattu
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Đọc tiếp các dòng kế
    FETCH NEXT FROM cur_Vattu
    -- Thực hiện đọc tiếp các dòng kế
    -- trong khi việc đọc dữ liệu thành công
END

--4. Đóng cursor
CLOSE cur_Vattu
DEALLOCATE cur_Vattu
```

CURSOR

- Để cập nhật giá trị dữ liệu cho cột TGNHAP (trị giá nhập) trong bảng PNHAP bằng cách duyệt qua từng phiếu nhập, tính ra trị giá nhập của từng phiếu căn cứ vào số lượng nhập và đơn giá nhập của từng vật tư trong bảng CTPNHAP, sau cùng cập nhật vào cột TGNHAP. Bạn sử dụng các lệnh như sau để thực hiện các hành động mô tả như trên :

```
--1.Khai báo biến cursor, các biến cục bộ
DECLARE @sSopn CHAR(4), @nTongtg MONEY
DECLARE cur_Pnhap CURSOR
FORWARD_ONLY
FOR
    SELECT SOPN
    FROM PNHAP

--2.Mở cursor
OPEN cur_Pnhap
--3.Đọc dữ liệu và cập nhật giá trị
WHILE 0 = 0
BEGIN
    FETCH NEXT FROM cur_Pnhap INTO @sSopn
    IF @@FETCH_STATUS <> 0
        BREAK
    SELECT @nTongtg = SUM(SLNHAP*DGNHAP)
    FROM CTPNHAP
    WHERE SOPN=@sSopn

    PRINT "Đang cập nhập phiếu nhập :" + @sSopn + " ..."
    UPDATE PNHAP
    SET TGNHAP = @nTongtg
    WHERE CURRENT OF cur_Pnhap
END

--4.Đóng cursor
CLOSE cur_Pnhap
DEALLOCATE cur_Pnhap
```

TRANSACTION



- Xét dữ liệu trong hai bảng Invoices và InvoiceItemLine tương ứng với InvoiceId = 12

Results		Messages									
InvoiceID	VendorID	InvoiceNumber	InvoiceDate	InvoiceTotal	PaymentTotal	CreditTotal	TermsID	InvoiceDueDate	PaymentDate	InvoiceInfo	
1	12	96	177271-001	2008-04-26 00:00:00	662.00	662.00	0.00	2	2008-05-16 00:00:00	2008-05-13 00:00:00	NULL

InvoiceID	InvoiceSequence	AccountNo	InvoiceLineItemAmount	InvoiceLineItemDescription	
1	12	1	580	50.00	D/Coco's
2	12	2	570	75.60	Kinko's
3	12	3	570	58.40	Office Max
4	12	4	540	478.00	Publishers Marketing

InvoiceTotal = Σ InvoiceLineItemAmount

InvoiceTotal = $\sum$ InvoiceLineItemAmount



- Câu lệnh Insert 1 và 2 thực thi thành công.
- Câu lệnh Insert 3 thực thi thất bại.

```
DECLARE @InvoiceID int
```

```
INSERT Invoices
VALUES (34, 'ZXA-080', '2008-08-30', 14092.59, 0, 0,
3, '2008-09-30', NULL)
```

```
SET @InvoiceID = @@IDENTITY
```

```
INSERT InvoiceLineItems
VALUES (@InvoiceID, 1, 160, 4447.23, 'HW Upgrade')
```

```
INSERT InvoiceLineItems
VALUES (@InvoiceID, 2, 167, 9645.36, 'OS upgrade')
```

TRANSACTION

- Để đảm bảo tính nhất quán của dữ liệu trong ví dụ trên.
 - Nếu một câu lệnh chèn trong ba câu lệnh chèn thực thi thất bại. Tất cả các câu lệnh chèn phải được hủy bỏ
- => Các câu lệnh chèn trên phải được đặt trong một giao dịch
- **Giao dịch**
 - Giao dịch là một nhóm thao tác cơ sở dữ liệu được kết hợp thành một đơn vị logic.

```
DECLARE @InvoiceID int
```

```
BEGIN TRY  
BEGIN TRAN
```

```
INSERT Invoices  
VALUES (34,'ZXA-080','2008-08-30',14092.59,  
0,0,3,'2008-09-30',NULL, NULL)
```

```
SET @InvoiceID = @@IDENTITY
```

```
INSERT InvoiceLineItems  
VALUES (@InvoiceID,1,160,4447.23,'HW upgrade')
```

```
INSERT InvoiceLineItems  
VALUES (@InvoiceID,2,167,9645.36,'OS upgrade')
```

```
COMMIT TRAN  
END TRY
```

```
BEGIN CATCH  
ROLLBACK TRAN  
END CATCH
```


TRANSACTION



- **THUỘC TÍNH CỦA TRANSACTION:** chuẩn gọi là ACID bao gồm 4 thuộc tính chuẩn.
 - **Atomicity** (Tính tự trị): Đảm bảo tất cả các thao tác, hành động trong phạm vi một đơn vị giao dịch là thành công hoàn toàn. Ngược lại, transaction sẽ bị dừng ngay ở thời điểm lỗi, và sẽ phục hồi quay ngược lại thời điểm chưa xảy ra sự thay đổi.
 - **Consistency** (Tính nhất quán): Đảm bảo tất cả các thao tác trên cơ sở dữ liệu được thay đổi sau khi giao dịch thành công và không xảy ra lỗi.
 - **Isolation** (Tính cô lập): Đảm bảo transaction này hoạt động độc lập so với transaction khác. Ví dụ như ở máy A đang thực hiện thì sẽ không liên quan tới giao dịch ở máy B.
 - **Durability** (Tính bền vững): Đảm bảo kết quả hoặc tác động của transaction vẫn luôn tồn tại, kể cả khi hệ thống xảy ra lỗi.

TRANSACTION



- HƯỚNG DẪN SỬ DỤNG TRANSACTION

```
SET XACT_ABORT ON
BEGIN TRAN
BEGIN TRY
    -- lệnh 1
    -- lệnh 2
    -- ...
COMMIT
END TRY
BEGIN CATCH
    ROLLBACK
    DECLARE @ErrorMessage VARCHAR(2000)
    SELECT @ErrorMessage = 'Lỗi: ' + ERROR_MESSAGE()
    RAISERROR(@ErrorMessage, 16, 1)
END CATCH
```

COMMIT để kết thúc transaction
ROLLBACK để quay lui transaction

SET XACT_ABORT ON tránh được rất nhiều rắc rối khi dùng transaction, nó xử lý gọn ghẽ các ngoại lệ kể cả các lỗi như connection timeout hay khi user hủy bỏ thực hiện.

TRANSACTION



- Kiểm soát Transaction trong SQL Server
- **Cách 1:** bằng cách kiểm tra biến toàn cục @@ERROR để xác định nếu có lỗi xảy ra thì thực hiện rollback. Điểm bất tiện của nó là sau mỗi câu lệnh SQL, ta phải thực hiện việc kiểm tra giá trị của biến này, để kịp thời rollback.

```
BEGIN TRANSACTION updatestaff;  
UPDATE aspnet.staff SET fk_department = 30  
WHERE pk_staff = 971;  
IF @@ERROR != 0 -- neu co loi xay ra, rollback  
BEGIN  
    PRINT 'rollback ne';  
    ROLLBACK TRAN updatestaff;  
END  
  
UPDATE aspnet.staff SET fk_department = NULL  
WHERE pk_staff = 971;  
IF @@ERROR != 0 -- neu co loi xay ra, rollback  
BEGIN  
    PRINT 'rollback ne';  
    ROLLBACK TRAN updatestaff;  
END  
ELSE  
    COMMIT TRANSACTION updatestaff;
```

TRANSACTION



- Kiểm soát Transaction trong SQL Server
- **Cách 2:** Sử dụng lệnh TRY...CATCH, hỗ trợ trên SQL Sever 2005, rất đơn giản và dễ dùng

```
BEGIN TRY
```

```
BEGIN TRANSACTION updatestaff;  
UPDATE aspnet.staff SET fk_department = 30  
WHERE pk_staff = 971;  
UPDATE aspnet.staff SET fk_department = NULL  
WHERE pk_staff = 971;
```

```
COMMIT TRANSACTION updatestaff;  
PRINT 'committed';  
END TRY  
BEGIN CATCH  
PRINT 'rollback ne';  
ROLLBACK TRAN updatestaff;  
END CATCH
```

@@ROWCOUNT



- @@ROWCOUNT: trả về số dòng bị ảnh hưởng bởi lệnh thực thi gần nhất

Ví dụ:

```
Update Employees set LastName = 'Brooke'
Where LastName ='Brook'
If(@@rowcount=0)
    begin
        print 'Không dòng nào được cập nhật'
        return
    end
```

```
BEGIN TRAN
```

```
DELETE Invoices
WHERE VendorID = 34
```

//Câu lệnh DELETE

```
IF @@ROWCOUNT > 1
```

```
    BEGIN
```

```
        ROLLBACK TRAN
```

```
        PRINT 'Deletions rolled back.'
```

```
    END
```

```
ELSE
```

```
    BEGIN
```

```
        COMMIT TRAN
```

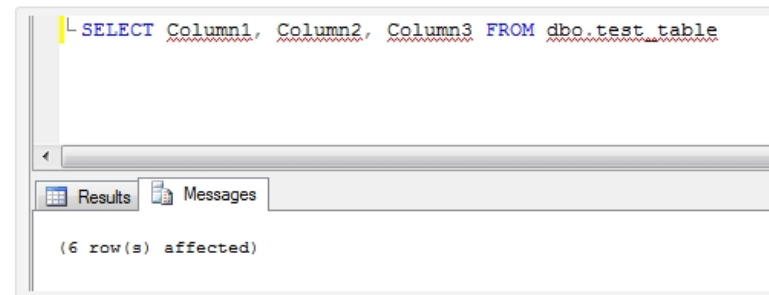
```
        PRINT 'Deletions committed to the database.'
```

```
    END
```

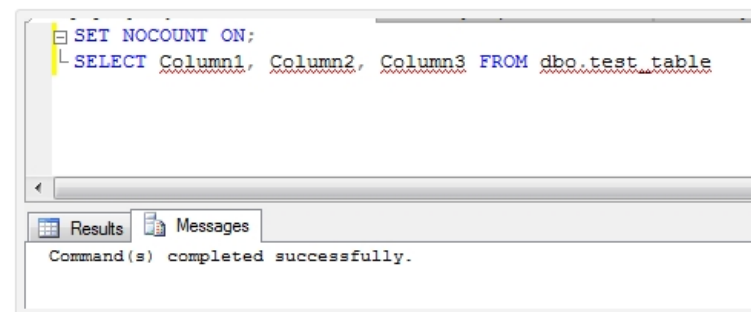
SET NOCOUNT ON



- By default the value of SET NOCOUNT is OFF in SQL Server, and for each SELECT, INSERT, UPDATE, and DELETE statement inside the stored procedure a message with the number of affected rows by that statement is returned.



- SET NOCOUNT ON will stop the message with the number of affected rows from being returned as part of the results.



SET NOCOUNT ON



By removing the extra overhead from the network it can greatly improve overall performance for your database and application
Using SET NOCOUNT ON

```
SET NOCOUNT ON;  
SELECT Column1, Column2, Column3 FROM dbo.test_table  
SELECT @@ROWCOUNT
```

Results Messages

	Column1	Column2	Column3
1	a	b	c
2	d	e	f
3	g	h	i
4	j	n	o
5	m	r	s
6	p		

(No column name)

1	6
---	---

@@TRANCOUNT



- Cứ một BEGIN TRANSACTION thì @@TRANCOUNT tăng lên 1 đơn vị

```
BEGIN TRANSACTION;

BEGIN TRY
    INSERT INTO T1 (C2, C3) VALUES ('A123456', 3);
    INSERT INTO T1 (C2, C3) VALUES ('A123457', 4);
    INSERT INTO T1 (C2, C3) VALUES ('A123458', 5);
    UPDATE T1 SET C3=4 WHERE C2='A123456';
END TRY

BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS ErrorNumber
        ,ERROR_SEVERITY() AS ErrorSeverity
        ,ERROR_STATE() AS ErrorState
        ,ERROR_PROCEDURE() AS ErrorProcedure
        ,ERROR_LINE() AS ErrorLine
        ,ERROR_MESSAGE() AS ErrorMessage;

    IF (@@TRANCOUNT > 0) ROLLBACK TRANSACTION;
END CATCH;

IF (@@TRANCOUNT > 0) COMMIT TRANSACTION;
GO

SELECT * FROM T1;
```

Lỗi phát sinh từ
câu lệnh này

ROLLBACK
TRANSACTION vì quá
trình cập nhật gặp lỗi

Nếu không gặp lỗi thì
phải COMMIT
TRANSACTION

@@TRANCOUNT



- Giao dịch lồng (nested transaction) là giao dịch được viết bên trong một giao dịch khác.
- Mỗi khi câu lệnh BEGIN TRAN được thực thi, hàm hệ thống @@TRANCOUNT được tăng thêm 1.
- Khi thực thi câu lệnh COMMIT TRAN
 - Nếu @@TRANCOUNT > 1, các thay đổi sẽ không được commit. Thay vào đó @@TRANCOUNT giảm đi 1.
 - Nếu @@TRANCOUNT = 1, mọi thay đổi đã được thực hiện trên CSDL trong suốt giao dịch sẽ được commit và @@TRANCOUNT được gán bằng 0.
- Câu lệnh ROLLBACK TRAN roll-back toàn bộ các giao dịch đang hoạt động và thiết lập giá trị cho @@TRANCOUNT về 0.

@@TRANCOUNT



```
BEGIN TRAN -- B tđ u giao dch 2
PRINT 'First Tran @@TRANCOUNT: ' + CONVERT(varchar, @@TRANCOUNT)
DELETE Invoices
    BEGIN TRAN -- B tđ u giao dch 2
        PRINT 'Second Tran @@TRANCOUNT: ' + CONVERT(varchar, @@TRANCOUNT)
        DELETE Vendors
    COMMIT TRAN          -- Câu l nh COMMIT này làm gi m giá tr @@TRANCOUNT.
                        -- Không COMMIT câu l nh 'DELETE Vendors'.
    PRINT 'COMMIT      @@TRANCOUNT: ' + CONVERT(varchar, @@TRANCOUNT)
ROLLBACK TRAN

PRINT 'ROLLBACK  @@TRANCOUNT: ' + CONVERT(varchar, @@TRANCOUNT)
PRINT ' '

DECLARE @VendorsCount int, @InvoicesCount int

SELECT @VendorsCount = COUNT (*) FROM Vendors
SELECT @InvoicesCount = COUNT (*) FROM Invoices

PRINT 'Vendors Count: ' + CONVERT (varchar , @VendorsCount)
PRINT 'Invoices Count: ' + CONVERT (varchar , @InvoicesCount)
```

A screenshot of a SQL Messages window. It displays the output of the SQL script, including transaction counts and row counts for DELETE operations. The window has a title bar that says "Messages".

```
Messages
First Tran @@TRANCOUNT: 1
(115 row(s) affected)
Second Tran @@TRANCOUNT: 2
(126 row(s) affected)
COMMIT      @@TRANCOUNT: 1
ROLLBACK    @@TRANCOUNT: 0

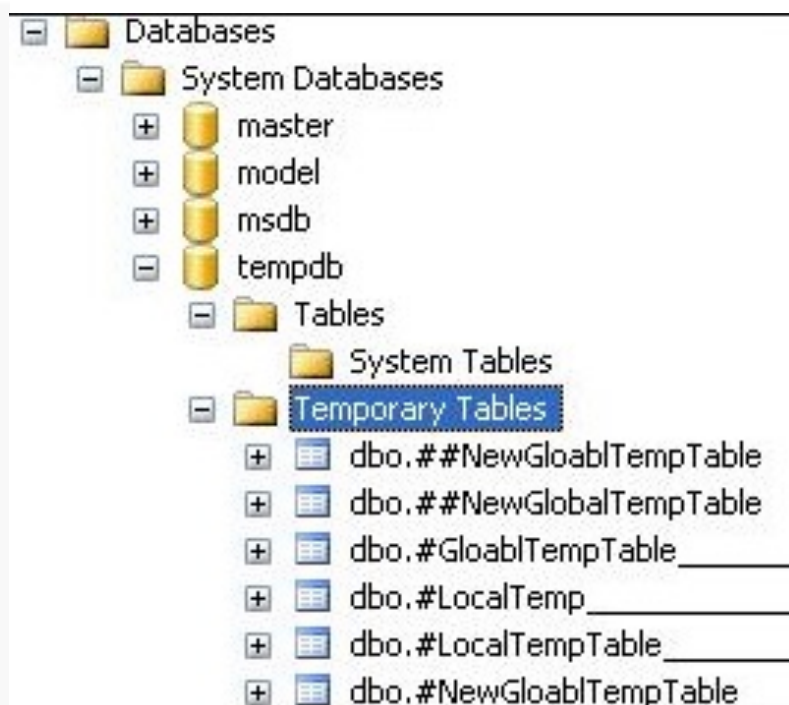
Vendors Count: 126
Invoices Count: 115
```

Bảng tạm & biến tạm

```
--tạo bảng trực tiếp
CREATE TABLE #t1 (C VARCHAR(50))

--vừa tạo bảng và thêm dữ liệu vào
SELECT C
INTO #t2
FROM dbo.SomeTable

--khai báo biến kiểu bảng
DECLARE @t TABLE (C VARCHAR(50))
INSERT INTO @t SELECT 1
SELECT * FROM @t
```



```

USE Northwind
GO
DECLARE @CustomerCopy TABLE
(
    CustomerID VARCHAR(10),
    ContactName VARCHAR(50),
    City VARCHAR(50),
    Country VARCHAR(50)
)

INSERT @CustomerCopy
(
    CustomerID,
    ContactName,
    City,
    Country
)
SELECT CustomerID, ContactName, City, Country FROM Customers

SELECT * FROM @CustomerCopy
UPDATE @CustomerCopy SET City='NEW YORK', Country='USA' WHERE CustomerID='ALFKI'
SELECT * FROM @CustomerCopy

```

Messages

Msg 1087, Level 16, State 1, Line 1
Must declare the table variable "@CustomerCopy".

Chạy cả script thì OK
Chạy từng phần báo lỗi

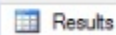


```
-- Creating Local Temp Tables
CREATE TABLE #LocalTemp
(
    [ID] [int] IDENTITY(1,1) NOT NULL,
    [FirstName] [nvarchar](255) NULL,
    [LastName] [nvarchar](255) NULL,
    [Occupation] [nvarchar](255) NULL,
    [YearlyIncome] [float] NULL,
    [Sales] [float] NULL
)

-- Inserting Values into Local SQL Temp Table
INSERT INTO #LocalTemp (
    [FirstName], [LastName], [Occupation], [YearlyIncome], [Sales])
VALUES ('Tutorial', 'Gateway', 'Education', 10000, 200)
,('Imran', 'Khan', 'Skilled Professional', 15900, 100)
,('Doe', 'Lara', 'Management', 15000, 60)
,('Ramesh', 'Kumar', 'Professional', 65000, 630)

-- Selecting Values From Local SQL Temp Table
SELECT [FirstName], [LastName], [Occupation], [YearlyIncome], [Sales] FROM #LocalTemp
```

100 %



Results



Messages

©tutorialgateway.org

	FirstName	LastName	Occupation	YearlyIncome	Sales
1	Tutorial	Gateway	Education	10000	200
2	Imran	Khan	Skilled Professional	15900	100
3	Doe	Lara	Management	15000	60
4	Ramesh	Kumar	Professional	65000	630



THANK YOU

Dr. Tran Anh Tuan, Faculty of Mathematics and Computer Science, University of
Science, HCMC